UNIT 6: DIVIDE AND CONQUER; INTEGER MULTIPLICATION ALGORITHM

The Karatsuba Algorithm

Ivan Temesvari
Distinguished Professor
Oakton College

Introduction

- Problem: Multiplying two large integers efficiently
- Traditional Approach: Grade-school multiplication algorithm (O(n^2) time complexity)
- Divide and Conquer Paradigm:
  - Breaking a problem into smaller subproblems
  - Solving subproblems recursively
  - Combining solutions to get the final answer
- Karatsuba Algorithm: A divide-and-conquer algorithm for integer multiplication with improved time complexity

The Karatsuba Algorithm

- Key Idea: Reduce the number of multiplications from 4 to 3 in the divide-and-conquer approach
- Steps:
  1. Split the input integers x and y into two halves:
     - $x = a * 10^{(n/2)} + b$
     - $y = c * 10^{(n/2)} + d$
  2. Compute three products recursively:
     - ac
     - bd
     - $(a + b) * (c + d)$
  3. Combine the results:
     - $x * y = ac * 10^n + [(a + b) * (c + d) - ac - bd] * 10^{(n/2)} + bd$

Recurrence Relation

- Let $T(n)$ be the time complexity of multiplying two n-digit numbers using the Karatsuba algorithm.
- Base Case: $T(1) = O(1)$ (multiplication of single-digit numbers)
- Recursive Case:
  - 3 recursive calls on numbers of size $n/2$
  - Some additions and subtractions, which take $O(n)$ time
  - $T(n) = 3T(n/2) + O(n)$

Analyzing the Recurrence Relation

- Apply the Master Theorem:
  - $a = 3$, $b = 2$, $f(n) = O(n)$
  - $\log_b(a) = \log_2(3) \approx 1.58$
  - $f(n) = O(n) = O(n^{\log_b(a) - \varepsilon})$ where $\varepsilon \approx 0.58$
  - This falls under Case 1 of the Master Theorem
- Conclusion: $T(n) = \Theta(n^{\log_b(a)}) = \Theta(n^{1.58})$

Comparison with Traditional Multiplication

- Traditional Multiplication: $O(n^2)$
- Karatsuba Algorithm: $O(n^{1.58})$
- Improvement: Significant for large values of n

Code Example (Pseudocode)

```
FUNCTION karatsuba(x, y)
    IF x < 10 OR y < 10 THEN
        RETURN x * y

    n = max(number of digits in x, number of digits in y)
    n_half = n / 2

    a = x / 10^n_half    // Left half of x
    b = x % 10^n_half    // Right half of x
    c = y / 10^n_half    // Left half of y
    d = y % 10^n_half    // Right half of y

    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    ad_plus_bc = karatsuba(a + b, c + d) - ac - bd

    RETURN (ac * 10^n) + (ad_plus_bc * 10^n_half) + bd
```

Case Studies
Case Study 1: Cryptography

- **Problem**: Secure communication and data protection require operations on very large numbers (e.g., RSA encryption, key generation).
- **Solution**: Karatsuba algorithm for fast multiplication of large integers involved in cryptographic computations.
- **Benefits**:
  - Speeds up encryption and decryption processes
  - Enables handling of larger keys for enhanced security.

Case Studies
Case Study 2: Scientific Computing

- **Problem**: Scientific simulations and numerical computations often involve high-precision arithmetic with very large numbers.
- **Solution**: Libraries implementing Karatsuba algorithm (or its variants) for efficient multiplication in scientific computing software.
- **Benefits**:
  - Accelerates computations involving large numbers
  - Enables handling of higher precision calculations.

Case Studies
Case Study 3: Computer Algebra Systems

- **Problem**: Symbolic computations and manipulations in computer algebra systems (e.g., Mathematica, Maple) often require operations on polynomials with large coefficients.
- **Solution**: Karatsuba-like algorithms for fast polynomial multiplication, improving the efficiency of symbolic computations.
- **Benefits**:
  - Enables handling of more complex symbolic expressions
  - Faster execution of algebraic operations

Conclusion

- **Key Takeaways**
  - Karatsuba algorithm is a faster approach for multiplying large integers
  - Employs the divide-and-conquer strategy
  - Reduces the number of multiplications, leading to improved time complexity
  - Recurrence relations are crucial for analyzing the algorithm's efficiency